

Extreme Subgroup Effects Under a Uniform Treatment Benefit

A Simulation Study Based on Observed Trial Data

Larry F. León

2026-02-23

Table of contents I

- 1 Motivation & Overview
- 2 Simulation Workflow
- 3 Part I — Building the Null DGM
- 4 Part II — Censoring Diagnostics & Calibration
- 5 Part III — Generating Synthetic Trials
- 6 Part IV — Simulation Results
- 7 Results — Forest Plots
- 8 Key Findings
- 9 Conclusions

Section 1

Motivation & Overview

The Core Question

Subgroup analysis in regulatory submissions

When many subgroups are examined in a single trial, small subgroups can produce hazard ratio estimates that look dramatic *purely by chance*.

This study asks directly

Under a DGM with a **perfectly uniform treatment benefit** (true HR = 0.70 for every patient), how often do standard Cox model analyses produce extreme subgroup results?

- Clinical subgroups of a given N are statistically indistinguishable from meaningless *random* groupings of the same size
- The **random** benchmarks random combinations and should fluctuate around overall results with “extremeness” due to variability of small samples as opposed to true subgroup effects

Setting: Uniform Treatment Effect

The DGM is built from a real trial (GBSG breast cancer, $N = 686$) using `generate_aft_dgm_flex()`.

Key feature — constant log hazard ratio

$$\psi^0(\mathbf{L}) = \log(\text{HR}_{\text{true}}) \quad \text{for all } \mathbf{L}.$$

- True overall HR = 0.70 (moderate benefit)
- Every subgroup shares *exactly* this HR
- Any apparent subgroup finding is a false positive by construction

Spline parameterisation

`log.hrs = log(c(0.70, 0.70, 0.70))` — uniform HR at biomarker values 0, knot k , and upper anchor ζ .

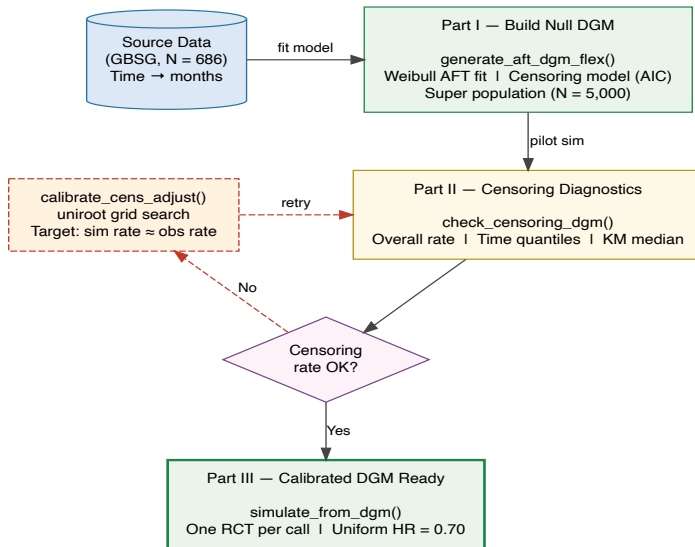
Section 2

Simulation Workflow

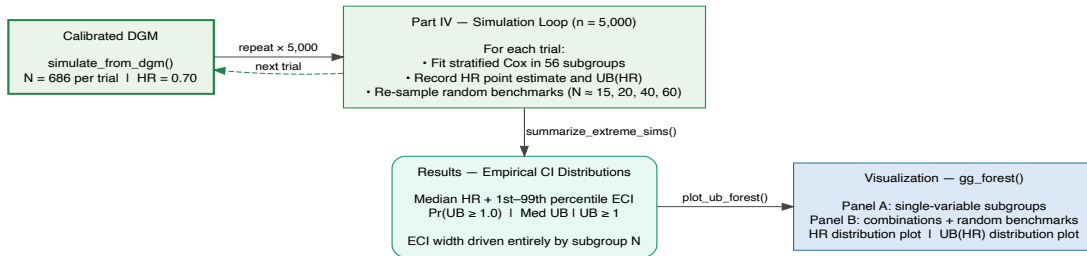
Pipeline Overview

Step	Function	Purpose
1	—	Convert source data to consistent time scale
2	<code>generate_aft_dgm_flex()</code>	Fit Weibull AFT; build super population
3	<code>check_censoring_dgm()</code>	Verify censoring matches source data
4	<code>calibrate_cens_adjust()</code>	Find <code>cens_adjust</code> via <code>uniroot</code>
5	<code>simulate_from_dgm()</code>	Draw synthetic trials from calibrated DGM
6	<code>coxph()</code> loop	Fit Cox in 56 subgroups; record HR and UB(HR)
7	<code>summarize_extreme_sims()</code>	Compute all summary statistics
8	<code>plot_ub_forest()</code>	Visualise ECI distributions

Workflow: DGM Setup & Calibration



Workflow: Simulation Loop & Results



Subgroup Definitions (1/3): ITT, Clinical, Continuous

#	Subgroup	Definition
ITT		
1	All Patients	All subjects (<code>flag_itt == 1</code>)
Clinical		
2	Post-menopausal	<code>meno == 1</code>
3	Pre-menopausal	<code>meno == 0</code>
4	Grade 3	<code>grade == 3</code>
5	Grade 1/2	<code>grade != 3</code>
Continuous		
6	Age (young)	<code>age < median</code>
7	Age (older)	<code>age > median</code>
8	Age ≤ 50	<code>age ≤ 50</code>
9	Age > 50	<code>age > 50</code>
10	Tumour size (small)	<code>size < median</code>
11	Tumour size (large)	<code>size > median</code>
12	Node-negative	<code>nodes == 0</code>
13	Nodes 1–3	<code>0 < nodes ≤ 3</code>

Subgroup Definitions (2/3): Two-Way Interactions

#	Subgroup	Definition
Meno × Grade		
19	Pre-meno / Grade 3	meno==0 & grade==3
20	Post-meno / Grade 3	meno==1 & grade==3
21	Pre-meno / Grade 1–2	meno==0 & grade!=3
22	Post-meno / Grade 1–2	meno==1 & grade!=3
Meno × Age		
23	Pre-meno / Age ≤ 50	meno==0 & age ≤ 50
24	Pre-meno / Age > 50	meno==0 & age>50
25	Post-meno / Age (young)	meno==1 & age med
26	Post-meno / Age (older)	meno==1 & age>med
Meno × ER		
27	Pre-meno / ER-low	meno==0 & er ≤ 20
28	Pre-meno / ER-high	meno==0 & er>20
29	Post-meno / ER-low	meno==1 & er ≤ 20
30	Post-meno / ER-high	meno==1 & er>20
Grade × Nodes		
31	Grade 3 / Node-neg	grade==3 & nodes==0
32	Grade 3 / Node-pos	grade==3 & nodes>0
33	Grade 1–2 / Node-neg	grade!=3 & nodes==0

Subgroup Definitions (3/3): Random Benchmarks

#	Subgroup	Approx N	Construction
53	random60	~60	Draw 60 random indices from trial
54	random40	~40	First 40 of the random60 indices
55	random20	~20	First 20 of the random60 indices
56	random15	~15	First 15 of the random60 indices

Re-sampled every simulation. Nested: random15 random20 random40 random60. No clinical meaning — true HR = 0.70 by construction.

Section 3

Part I — Building the Null DGM

Fitting `generate_aft_dgm_flex()`

- Fits a **Weibull AFT model** to source data (GBSG)
- Generates a large *super population* ($N_{\text{super}} = 5000$) from which all future trials are drawn
- `model = "null"` — treatment effect identical for every patient
- Censoring model estimated by AIC selection (Weibull / log-normal)

R code

```
dgm_null <- generate_aft_dgm_flex(  
  data = gbsg, model = "null",  
  subgroup_vars = NULL,      n_super = 5000)
```

Section 4

Part II — Censoring Diagnostics & Calibration

- Credible results require censoring in synthetic trials to match source data
- `check_censoring_dgm()` compares three quantities:
 - ① **Overall censoring rate** vs `rate_tol` (default 10 pp)
 - ② **Censoring-time quantiles** among censored subjects
 - ③ **KM median censoring time** vs `median_tol` (default 25%)

Calibrating cens_adjust

`calibrate_cens_adjust()` uses `uniroot` to find the log-scale shift that equates a target statistic between reference and simulated data.

Objective function

At each candidate `cens_adjust`: (1) generate synthetic subjects, (2) extract target metric, (3) return `sim_metric - ref_metric`.

Target	Equates
"rate"	Overall censoring rate
"km_median"	KM-based median censoring time

Monotonicity: larger `cens_adjust` $\Rightarrow$ longer censoring times.

Section 5

Part III — Generating Synthetic Trials

`simulate_from_dgm()`

With the null DGM calibrated, each simulated trial is a single function call:

- ① Samples from super population with replacement
- ② Assigns treatment (1:1 randomisation)
- ③ Generates latent survival times from the AFT model
- ④ Applies calibrated censoring model
- ⑤ Truncates at `analysis_time`

Under the null DGM: `flag_harm` $\equiv$ 0 for every subject.

Section 6

Part IV — Simulation Results

Subgroups Under Study (56 total)

Group	Examples
ITT	All Patients ($N \approx 686$)
Clinical	Post-/Pre-menopausal, Grade
Continuous	Age, Size, Nodes, PGR, ER
Interaction	Meno $\times$ Grade, ER, Age; Grade $\times$ Nodes, ER, PGR

Group	Examples
3-way	Pre-meno/Young/ER-low, ...
Size $\times$ Nodes	Large/Node-pos, ...
Random	random60, 40, 20, 15

Key benchmark

random* subgroups have **no clinical meaning**.
Re-sampled each simulation.

HR point estimate distribution

- Median + 1st–99th percentile (ECI)
- Under truth, median ≈ 0.70 for every subgroup
- ECI width driven by subgroup N

Upper confidence bound $UB(HR)$

- Upper end of the Cox 95% CI
- $\Pr(UB < 1.0)$: power to show benefit
- Med $UB \mid UB \geq 1$: worst-case for the safety reviewer

Simulation: 1000 trials completed

- **1000** simulated trials from the null DGM (true HR = 0.70)
- **56** subgroups evaluated per trial
- Subgroup sizes range from $N \approx 15$ (random15) to $N \approx 686$ (ITT)

Re-use with new results

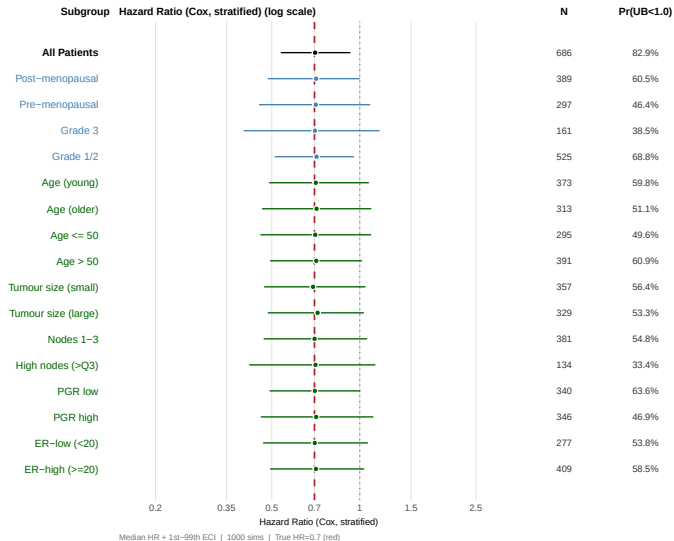
```
res <- load_sim_results("new_file.rds")  
S <- summarize_extreme_sims(res$sim_hrs, res$sim_ubs, res$sim_ns, res$subgroups)
```

All downstream tables and plots update automatically.

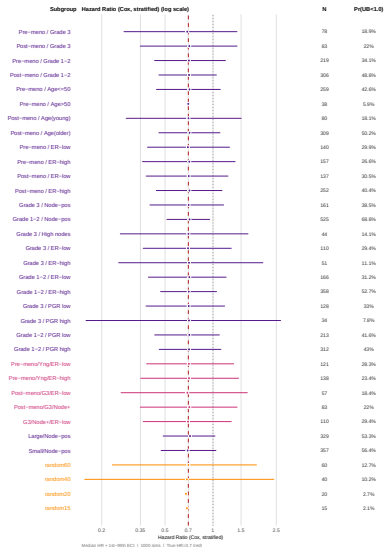
Section 7

Results — Forest Plots

HR Distributions: Single-Variable Subgroups



HR Distributions: Combinations + Random Benchmarks



UB(HR) Distributions: Single-Variable Subgroups { .shrink=5 }

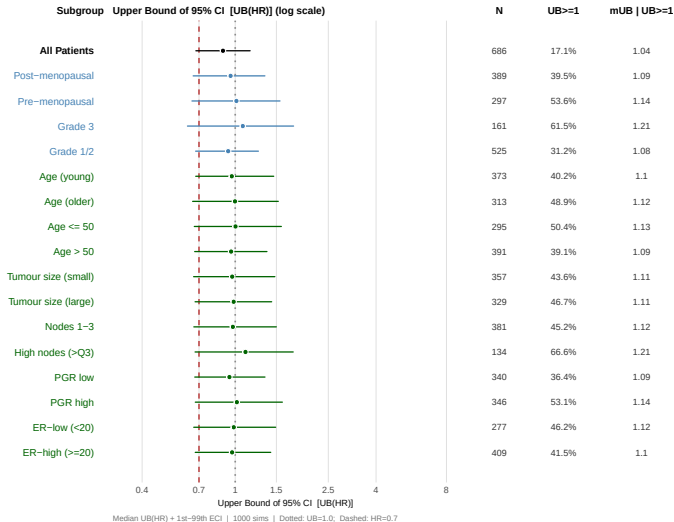


Figure 5: UB(HR) single-variable subgroups. Dotted: UB=1.0; Dashed: true HR=0.70

UB(HR) Distributions: Combinations + Random Benchmarks {.shrink=5}

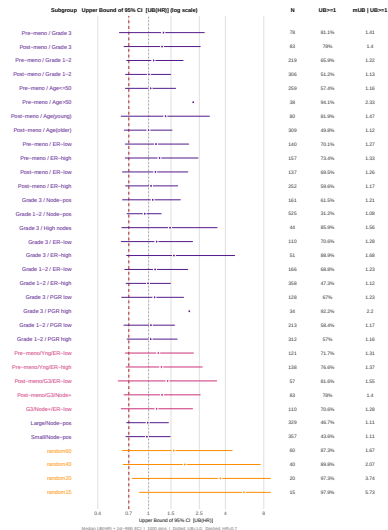


Figure 6: UB(HR) combination and random benchmark subgroups

Section 8

Key Findings

HR Point Estimates

Large subgroups ($N > 200$)

Median HR tracks truth (≈ 0.70) closely; ECI is narrow.

Small subgroups ($N \lesssim 40$)

- ECI for random15/random20 spans ~ 0.25 to > 1.0
- Clinical subgroups of similar size show **indistinguishable** ECI widths
- Clinical rationale provides **no statistical protection**

Clinical subgroups of a given N are statistically indistinguishable from meaningless random groupings of the same size.

Upper Bounds UB(HR)

The upper-bound distribution

- random40 ($N \approx 40$): 1% of trials produce $UB(HR) > 3.0-4.0$
- random15 ($N \approx 15$): values above 5.0 not rare
- A reviewer seeing $UB(HR) = 3.5$ in 15 patients cannot distinguish chance from a genuine safety signal

Illustrative 99th pctl of UB(HR)

Subgroup	P99 UB
ITT ($N \approx 686$)	< 1.1
$N \approx 200$	~ 1.5
$N \approx 60$	~ 2.5
random40	~ 3.5
random20	~ 5.0
random15	> 5.0

Cox Model Works Correctly

What the simulation confirms

- ① Cox model is **unbiased** — median HR ≈ 0.70 for every subgroup
- ② Confidence intervals have correct **coverage**
- ③ Variability driven **entirely by sample size**

The interpretive hazard

- $UB(HR) = 3.5$ in a 15-patient subgroup is *expected* under uniform benefit
- Without simulation-based calibration, no way to quantify how “extreme” a result is
- The `random*` benchmarks provide exactly this calibration

Section 9

Conclusions

- ① **Small subgroups produce extreme results by chance.** Under uniform $HR = 0.70$, subgroups with $N \lesssim 40$ regularly produce $UB(HR) > 3.0$.
- ② **Clinical rationale does not protect against noise.** Clinical subgroups have the *same* HR/UB distribution as random benchmarks of that size.
- ③ **The Cox model is working correctly.** Wide CIs in small subgroups are accurate — they reflect genuine uncertainty.
- ④ **Simulation-based calibration is essential.** Without a null-DGM benchmark, reviewers cannot quantify “extreme” for a given subgroup size.
- ⑤ **The forestsearch package** provides a complete, reproducible workflow.

Thank You

forestsearch R package

<https://larry-leon.github.io/forestsearch/>

```
pak::pak("larry-leon/forestsearch")
```

Section 10

Appendix

New-Results Workflow

```
source("summarize_extreme_sims.R")

# Option A: Load existing results
res <- load_sim_results("results/extreme_sims_5000.rds")

# Option B: After running a new simulation loop
save_sim_results(sim_hrs, sim_ubs, sim_ns, subgroups,
                 file = "results/extreme_sims_new.rds",
                 hr_true = 0.70,
                 metadata = list(dgm_seed = 99, n_per_trial = 686))

# Summarize (works with either option)
S <- summarize_extreme_sims(res$sim_hrs, res$sim_ubs,
                           res$sim_ns, res$subgroups)

# All plots driven by S
plot_hr_forest(S, panel = "single")
plot_hr_forest(S, panel = "combo")
plot_ub_forest(S, panel = "single") # <-- ub-forest-single
```

summarize_extreme_sims() API

Field	Description
hr_q, ub_q	Quantile matrices [$n_{sg} \times 3$]: 1st, median, 99th
mean_n	Mean subgroup N across simulations
pr_ub_lt1, pr_ub_ge1	Power / failure rate per subgroup
cond_ub	Conditional UB quantiles given $UB \geq 1$
results_tbl	Formatted data.frame for printing
ok, ok_ub	Logical: valid subgroups
idx_hr_single, idx_ub_combo, ...	Index vectors for forest panels
cat_ok, cat_cols	Category labels and colour map
lab_cmed_ub	Pre-formatted conditional median labels

Comparing Two Simulation Runs

```
S_old <- summarize_extreme_sims(...) # e.g. n_sims = 1000
S_new <- summarize_extreme_sims(...) # e.g. n_sims = 5000

comp <- compare_extreme_sims(S_old, S_new,
                             metric = "both",
                             label1 = "1000 sims",
                             label2 = "5000 sims")

print(comp, row.names = FALSE)
# Subgroup | N | MedHR | Delta_MedHR | PrUBge1 | Delta_PrUBge1 | MedUBcond
```

Useful for verifying convergence, comparing DGM variants, or assessing sensitivity to censoring calibration.

Helper Utilities

```
# Optimal figure height
fh <- forest_fig_height(S, panel = "combo", type = "ub",
                        row_height = 0.45, overhead = 1.5)

# Save / load round-trip
save_sim_results(sim_hrs, sim_ubs, sim_ns, subgroups,
                  file = "results/my_run.rds", hr_true = 0.70,
                  metadata = list(cens_adjust = 0.42))

res <- load_sim_results("results/my_run.rds")
# prints: Loaded: ... (5000 sims x 56 subgroups, saved ...)

# Print method
print(S) # shows n_sims, n_sg, hr_true, then results_tbl
```